

MODULE III

Module III - part 2

interfacing sensors with Arduino-humidity, temperature, water detector, PIR, ultrasonic sensor, LDR, Interfacing Push Button Switch-Pull Up & Pull-Down Connection.

Interfacing our Arduino board with different sensors.

We will discuss the following sensors –

- Humidity sensor (DHT22)
- Temperature sensor (LM35)
- Water detector sensor (Simple Water Trigger)
- PIR SENSOR
- ULTRASONIC SENSOR
- LDR

1. Humidity Sensor (DHT22)

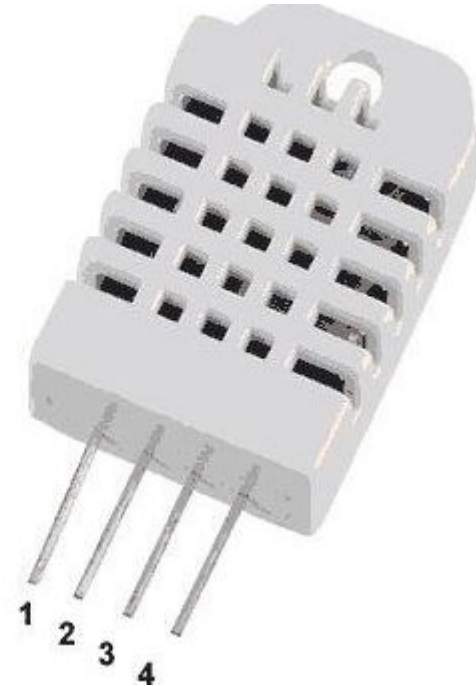
The **DHT22** is a popular sensor for measuring **temperature and humidity** with good accuracy.

The DHT-22 (also named as AM2302) is a digital-output, relative humidity, and temperature sensor.

It uses a capacitive humidity sensor and a thermistor to measure the surrounding air, and sends a digital signal on the data pin.

DHT22 pins	
1	VCC
2	DATA
3	NC
4	GND

Nc — Not connected



DHT22 (Temperature & Humidity Sensor) – Uses

1. Smart Thermostats

- Reads indoor temperature/humidity for climate control.

2. Greenhouse Monitoring

- Maintains ideal humidity and temperature for plant growth.

3. Weather Stations

- Core sensor for tracking environmental conditions outdoors.

4. Dehumidifier/AC Control

- Turns on/off based on current room humidity levels.

5. Data Logging Projects

- Collects and stores environmental data over time for analysis.

Arduino Code for DHT22:

```
#include "DHT.h"

#define DHTPIN 2      // DHT22 data pin connected to
digital pin 2
#define DHTTYPE DHT22 // DHT22 (AM2302)

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(9600);
  dht.begin();
}

void loop() {
  float humidity = dht.readHumidity();
  float temperature = dht.readTemperature(); // Celsius
  // float tempF = dht.readTemperature(true); // Fahrenheit

```

```
    if (isnan(humidity) || isnan(temperature)) {
      Serial.println("Failed to read from DHT
        sensor!");
      return;
    }

    Serial.print("Humidity: ");
    Serial.print(humidity);
    Serial.print(" %\t");

    Serial.print("Temperature: ");
    Serial.print(temperature);
    Serial.println(" °C");

    delay(2000); // DHT22 has a 2-second minimum
    sample rate
}
```

Output Example:

Humidity: 56.70 %

Temperature: 24.30 °C

Why You Need the DHT Library: *#include "DHT.h"*

The **DHT22 sensor** uses a special one-wire protocol to communicate — and that protocol is a bit complex to handle manually.

The **Adafruit DHT sensor library**:

- Handles all the timing and data decoding for you
- Makes it super easy to get `readTemperature()` and `readHumidity()` with just one line of code

2. Temperature Sensor :

The Temperature Sensor LM35 series are precision integrated-circuit temperature devices with an output voltage linearly proportional to the Centigrade temperature.

The LM35 device has an advantage over linear temperature sensors calibrated in Kelvin, as the user is not required to subtract a large constant voltage from the output to obtain convenient Centigrade scaling. The LM35 device does not require any external calibration or trimming to provide typical accuracies of $\pm 0.1^\circ\text{C}$ at room temperature and $\pm 0.5^\circ\text{C}$ over a full -55°C to 150°C temperature range.

LM35 Temperature Sensor – Uses

1. Room Temperature Monitoring

- Measure ambient temperature in a home, greenhouse, or classroom.

2. Overheating Protection

- Shut down devices when temperatures exceed safe limits (e.g., in electronics or battery systems).

3. CPU Temperature Sensing

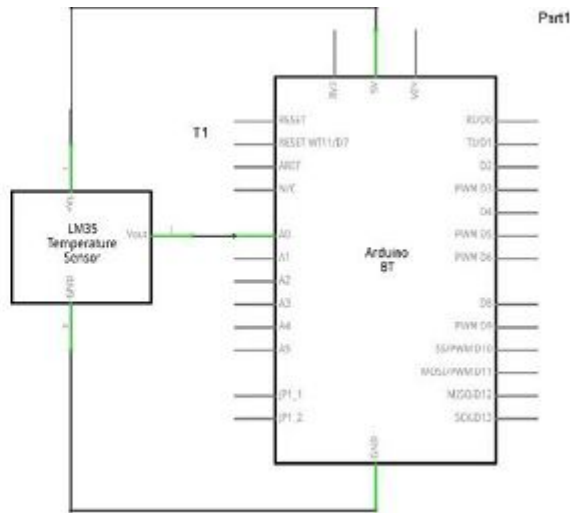
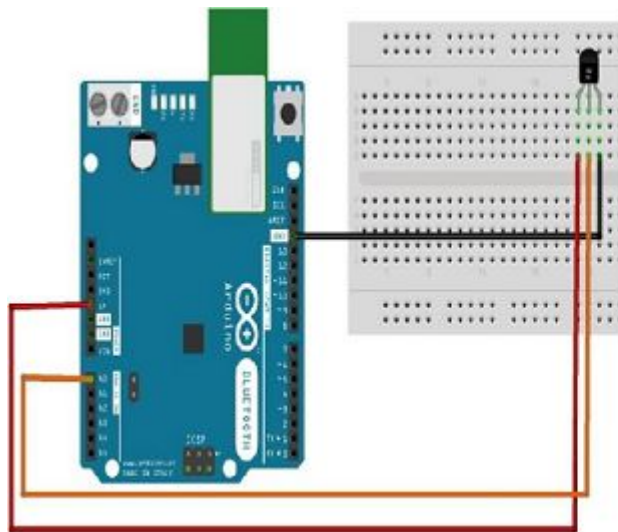
- Mounted near processors to monitor and manage heat.

4. Weather Stations

- Used for accurate outdoor temperature readings in DIY weather stations.

5. HVAC Systems

- Helps control heating/cooling systems automatically.



Arduino Code

```
float temp;  
int tempPin = 0;  
  
void setup() {  
  Serial.begin(9600);  
}  
  
void loop() {  
  temp = analogRead(tempPin);  
  // read analog volt from sensor and save to variable temp  
  temp = temp * 0.48828125;  
  // convert the analog volt to its temperature equivalent  
  Serial.print("TEMPERATURE = ");  
  Serial.print(temp); // display temperature value  
  Serial.print("°C");  
  Serial.println();  
  delay(1000); // update sensor reading each one second  
}
```

Reads from analog pin A0 (which is
`tempPin = 0`)

Converts the analog value to temperature
using:

`temp = temp * 0.48828125;`

Where **0.48828125** Comes From:

Analog range = 0 to 1023

Voltage range = 0 to 5V

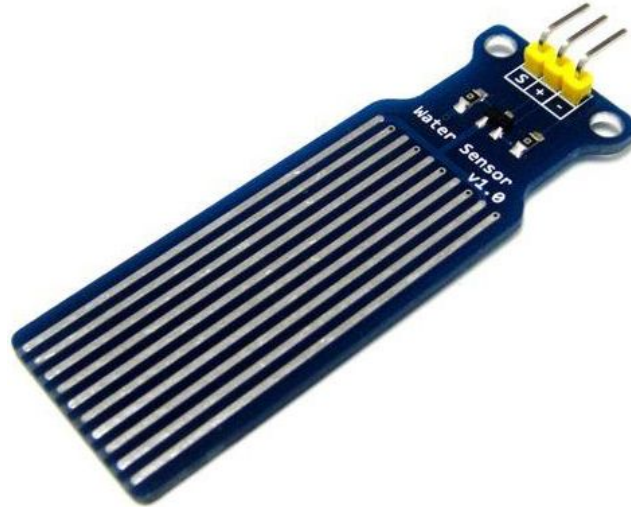
- So, $5V / 1024 \approx 0.00488 \text{ V per unit}$
- The LM35 sensor outputs 10mV per °C, so:

$$\begin{aligned} \text{temp} &= \text{analogRead} * (5V / 1024) / (10\text{mV}) \\ &= \text{analogRead} * 0.00488 / 0.01 \\ &= \text{analogRead} * 0.48828125 \end{aligned}$$

3. Arduino - Water Detector / Sensor

Water sensor brick is designed for water detection, which can be widely used in sensing rainfall, water level, and even liquid leakage.

Connecting a water sensor to an Arduino is a great way to detect a leak, spill, flood, rain, etc. It can be used to detect the presence, the level, the volume and/or the absence of water. While this could be used to remind you to water your plants, there is a better Grove sensor for that. The sensor has an array of exposed traces, which read LOW when water is detected.



Water Detector Sensor (Simple Water Trigger) – Uses

1. Water Leakage Alert System

- Detects leaks under sinks, tanks, or near pipes and triggers a buzzer/alarm.

2. Rain Detection for Automatic Windows

- Closes windows or skylights automatically when it starts raining.

3. Water Level Monitoring

- Detects if water reaches a certain height (e.g., sump pits or tanks).

4. Plant Soil Moisture Alert (Modified use)

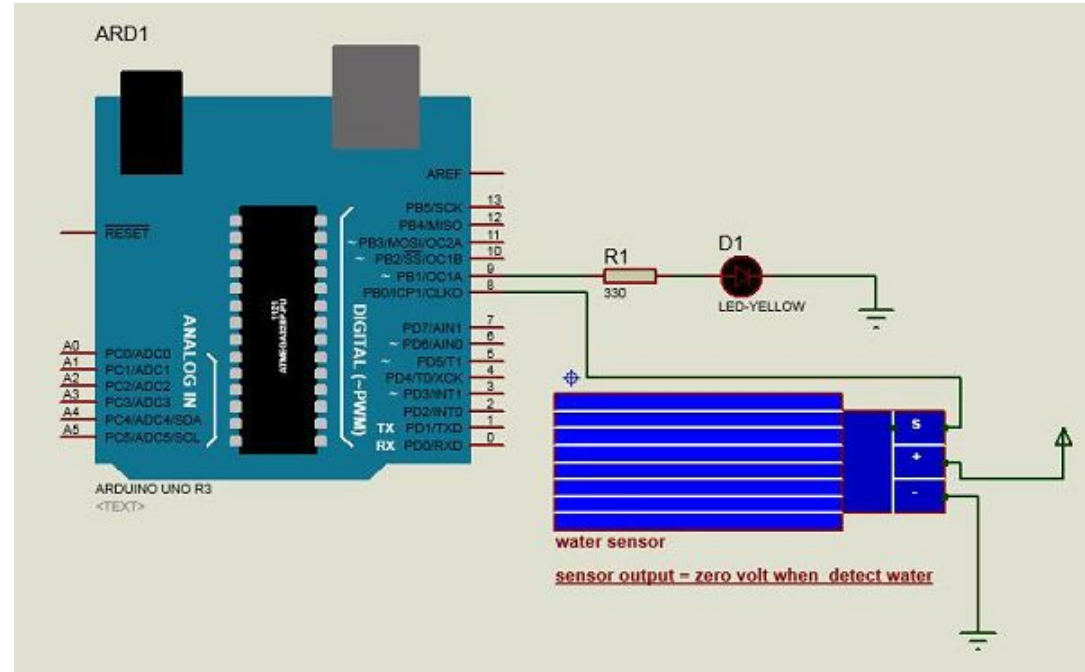
- Detects if the soil is too dry or wet (with slight calibration).

5. Smart Appliances Safety

- Auto-shutoff for machines like washing machines or dishwashers when water is detected in wrong areas.

Water sensor has three terminals - S, V_{out} (+), and GND (-). Connect the sensor as follows –

- Connect the $+V_s$ to +5v on your Arduino board.
- Connect S to digital pin number 8 on Arduino board.
- Connect GND with GND on Arduino.
- Connect LED to digital pin number 9 in Arduino board.



Code :

```
#define Grove_Water_Sensor 8 // Attach Water sensor to Arduino Digital Pin 8  
#define LED 9 // Attach an LED to Digital Pin 9 (or use onboard LED)  
  
void setup() {  
  pinMode(Grove_Water_Sensor, INPUT); // The Water Sensor is an Input  
  pinMode(LED, OUTPUT); // The LED is an Output  
}  
  
void loop() {  
  /* The water sensor will switch LOW when water is detected.  
  Get the Arduino to illuminate the LED and activate the buzzer  
  when water is detected, and switch both off when no water is present */  
  if( digitalRead(Grove_Water_Sensor) == LOW) {  
    digitalWrite(LED,HIGH);  
  }else {  
    digitalWrite(LED,LOW);  
  }  
}
```

When the sensor detects water, pin 8 on Arduino becomes LOW and then the LED on Arduino is turned ON.

4. PIR Sensor :

- PIR sensors allow you to sense motion.
- They are used to detect whether a human has moved in or out of the sensors range.
- They are commonly found in appliances and gadgets used at home or for businesses.
- They are often referred to as PIR, "Passive Infrared", "Pyroelectric", or "IR motion" sensors.

PIR (Passive Infrared) Sensor – Uses

1. Motion-Activated Lighting

- Turns lights on when motion is detected, ideal for rooms, hallways, or outdoor spaces.

2. Security/Intruder Alarms

- Triggers sirens or notifies the system when someone enters an area.

3. People Counting in Smart Buildings

- Detects room occupancy to manage lighting, ventilation, or AC.

4. Automatic Door Systems

- Detects approaching people to open doors (like in malls or elevators).

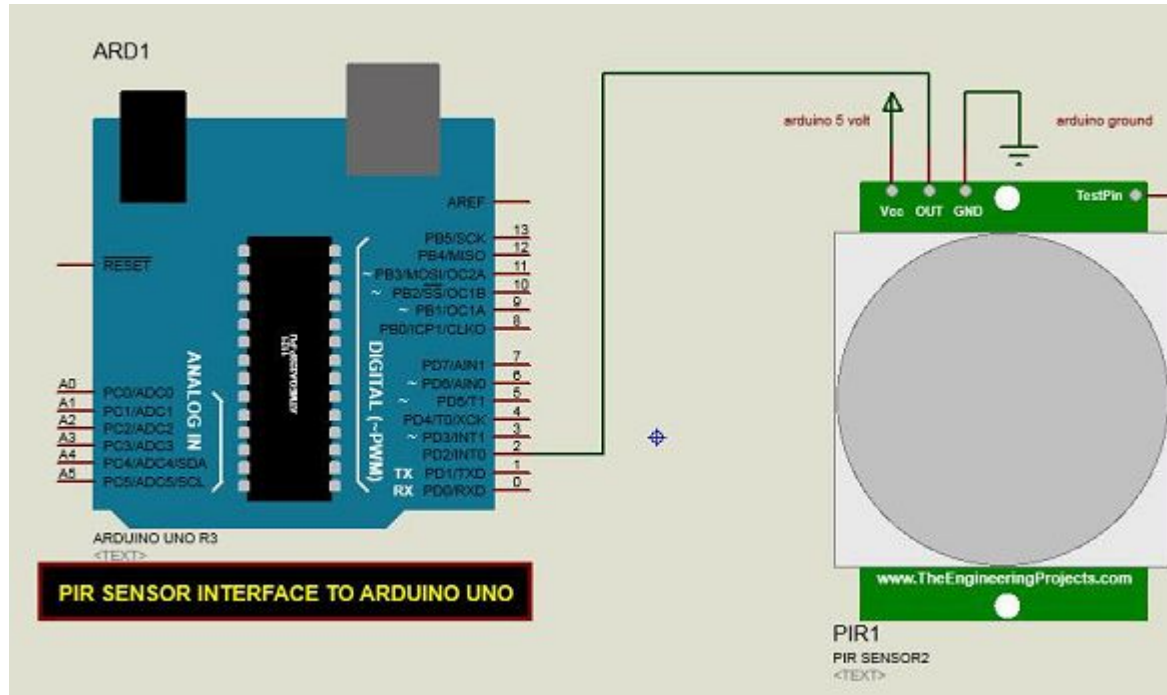
5. Energy Saving Systems

- Automatically powers down devices when no motion is detected for a while.



PIR sensor has three terminals - V_{cc} , OUT and GND. Connect the sensor as follows –

- Connect the $+V_{cc}$ to $+5v$ on Arduino board.
- Connect OUT to digital pin 2 on Arduino board.
- Connect GND with GND on Arduino.



Code :

```
const int pirPin = 2;    // PIR sensor output connected to digital pin 2
const int ledPin = 13;   // Built-in LED on most Arduino boards

void setup() {
  pinMode(pirPin, INPUT); // Set PIR pin as input
  pinMode(ledPin, OUTPUT); // Set LED pin as output
  Serial.begin(9600);     // Start serial communication for debugging
}

void loop() {
  int pirState = digitalRead(pirPin); // Read PIR sensor state

  if (pirState == HIGH) {
    digitalWrite(ledPin, HIGH);    // Turn on LED
    Serial.println("Motion detected!");
  } else {
    digitalWrite(ledPin, LOW);     // Turn off LED
    Serial.println("No motion.");
  }

  delay(500); // Wait for half a second before reading again
}
```

If motion is detected:

- LED turns **ON** (digitalWrite(ledPin, HIGH))
- Message printed: **"Motion detected!"**

If no motion:

- LED turns **OFF**
- Message printed: **"No motion."**

5. Ultrasonic Sensor

The HC-SR04 ultrasonic sensor uses SONAR to determine the distance of an object just like the bats do. It offers excellent non-contact range detection with high accuracy and stable readings in an easy-to-use package from 2 cm to 400 cm or 1 to 13 feet.

The operation is not affected by sunlight or black material, although acoustically, soft materials like cloth can be difficult to detect. It comes complete with ultrasonic transmitter and receiver module.



Ultrasonic Sensor – Uses

1. **Obstacle Detection for Robots**

- Helps robots or cars detect and avoid walls or objects.

2. **Distance Measurement Tools**

- Measures distance between the sensor and an object (used in parking sensors).

3. **Automatic Water Tank Filling**

- Detects water level in tanks and controls motor ON/OFF accordingly.

4. **Smart Trash Can Lid**

- Opens the lid when someone's hand comes near (contactless).

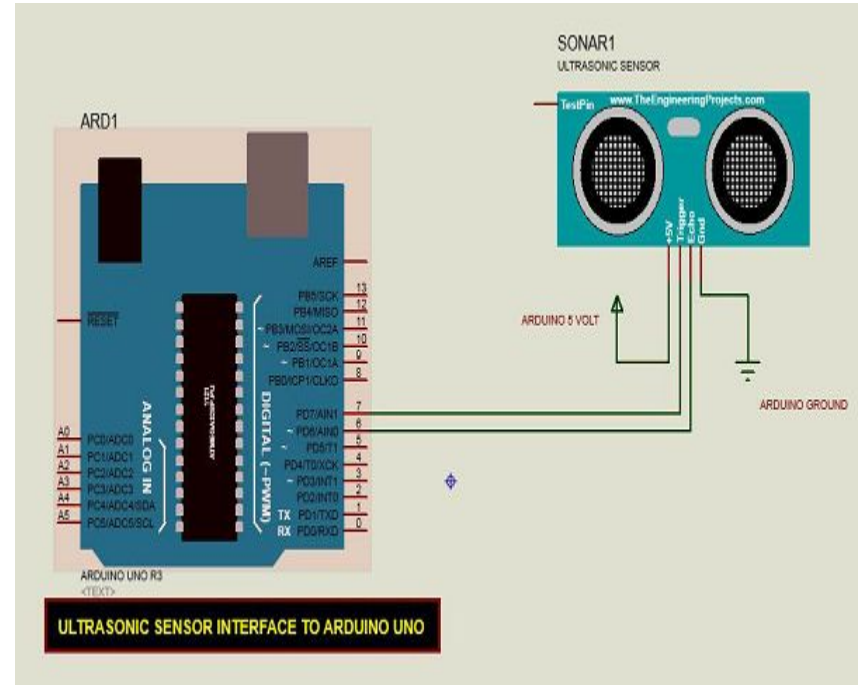
5. **Blind Spot Detection System**

- Used in vehicles to alert if something is within close range.

Procedure :

The Ultrasonic sensor has four terminals - +5V, Trigger, Echo, and GND connected as follows –

- Connect the +5V pin to +5v on your Arduino board.
- Connect Trigger to digital pin 7 on your Arduino board.
- Connect Echo to digital pin 6 on your Arduino board.
- Connect GND with GND on Arduino.



How it works....

1. Trigger the Sensor

- The **Trigger pin** is set HIGH for **10 microseconds**.
- This tells the sensor to **emit a sound pulse** at **40 kHz** (inaudible to humans).

2. Sound Travels

- The sensor sends out a **burst of sound waves** into the air.
- These waves travel forward at the **speed of sound** (~343 meters/second at room temperature).

3. Sound Hits an Object

- When the sound hits an object, it **bounces back** like an echo.

4. Echo Pin Detects Return

- The **Echo pin** goes HIGH while the echo is returning.
- You measure how long the **Echo pin stays HIGH** → this is the **round-trip time** of the sound wave.

5. Calculate Distance

- Since you know the **speed of sound**, and you now have **time**, you can calculate distance:

$$\text{Distance} = \frac{\text{Time} \times \text{Speed of Sound}}{2}$$

(You divide by 2 because the sound goes **to the object and back**.)

Arduino Code

```
const int pingPin = 7; // Trigger Pin of Ultrasonic Sensor
const int echoPin = 6; // Echo Pin of Ultrasonic Sensor
```

```
void setup() {
  Serial.begin(9600); // Starting Serial Terminal
}
```

```
void loop() {
  long duration, inches, cm;
  pinMode(pingPin, OUTPUT);
  digitalWrite(pingPin, LOW);
  delayMicroseconds(2);
  digitalWrite(pingPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(pingPin, LOW);
  pinMode(echoPin, INPUT);
```

```
  duration = pulseIn(echoPin, HIGH);
  inches = microsecondsToInches(duration);
  cm = microsecondsToCentimeters(duration);
  Serial.print(inches);
  Serial.print("in, ");
  Serial.print(cm);
  Serial.print("cm");
  Serial.println();
  delay(100);
}
```

```
long microsecondsToInches(long microseconds) {
  return microseconds / 74 / 2;
}
```

```
long microsecondsToCentimeters(long microseconds) {
  return microseconds / 29 / 2;
}
```

- The speed of sound in **inches per microsecond** is approximately:

$$1 \text{ inch} = 74 \mu\text{s} \text{ (for round trip)}$$

- So for **each inch**, the sound takes **~74 μs** to **go to the object and return**.

It takes about **29 microseconds** for sound to travel **1 cm** and back.

Feature	PIR Sensor	Ultrasonic Sensor
Detects	Motion (heat changes)	Distance to objects
Detects static object	✗ No	✓ Yes
Measures distance	✗ No	✓ Yes
Light/Dark sensitivity	✓ Works in both	✓ Works in both
Cost	\$ Cheap	\$\$ Moderate
Power usage	🔋 Very low	🔋🔋 Medium

6. LDR

- **LDR (Light Dependent Resistor)**
- It changes its **resistance based on light intensity**.
- More light = **less resistance**
- Less light = **more resistance**

Uses

1. **Automatic Street Lighting**
Turns ON street lights at night and OFF during the day.
2. **Brightness Control**
Adjusts screen brightness on phones/laptops based on ambient light.
3. **Solar Trackers**
Detects sunlight direction to align solar panels for max efficiency.
4. **Security Systems**
Detects light changes (e.g., someone passing by) to trigger alarms.
5. **Light-Controlled Fan or AC**
Activates appliances based on sunlight (e.g., fan turns on during daylight).



Light Condition	Resistance of LDR	Analog Output (0–1023)
Very bright light	Low (few hundred ohms)	High (800–1023)
Normal light	Medium (1k–10k ohms)	Mid (400–800)
Darkness	High (100k+ ohms)	Low (0–400)

Basic Arduino Code:

```
int ldrPin = A0; // Analog pin connected to the LDR/resistor junction
int ldrValue = 0;
```

```
void setup() {
  Serial.begin(9600);
}
```

```
void loop() {
  ldrValue = analogRead(ldrPin); // Read the analog value (0–1023)
  Serial.print("Light Intensity: ");
  Serial.println(ldrValue);      // Higher = more light

  delay(500);
}
```

Sample Output:

Light Intensity: 830 ← bright

Light Intensity: 102 ← dark

Arduino code that uses a Light Dependent Resistor (LDR) to control an LED.

```
const int ldrPin = A0;           // Pin connected to the LDR
const int ledPin = 9;           // Pin connected to the LED
const int threshold = 900;      // Threshold value

void setup() {
  pinMode(ledPin, OUTPUT);      // Set LED pin as output
  Serial.begin(9600);           // Start serial communication for debugging
}

void loop() {
  int ldrValue = analogRead(ldrPin); // Read the LDR value
  Serial.println(ldrValue);         // Print the value to the Serial Monitor
  // Check if the LDR value exceeds the threshold

  if (ldrValue > threshold) {
    digitalWrite(ledPin, LOW);      // Turn off the LED
  } else {
    digitalWrite(ledPin, HIGH);     // Turn on the LED
  }
  delay(100);                      // Small delay for stability
}
```

